

Quality requirements challenges in the context of large-scale distributed agile: An empirical study

Wasim Alsaqaf*, Maya Daneva, Roel Wieringa

University of Twente, Enschede, the Netherlands

ARTICLE INFO

Keywords:

Agile large-scale distributed projects
Requirements engineering
Quality requirements
Exploratory empirical research method
Interviews

ABSTRACT

Context: Engineering quality requirements in agile projects does not fit organically with agile methods. Despite the agile community acknowledges this, little empirical evidence has been published on this topic.

Objective: This exploratory qualitative interview-based study explicates the challenging situations experienced by practitioners in engineering the quality requirements in the context of large-scale distributed agile projects. Moreover, this study describes the practices that agile distributed teams currently use which could contribute by dealing with the identified challenges.

Method: The challenging situations and possible mitigation practices were studied from the perspective of 17 practitioners from large distributed agile project teams in six organizations in The Netherlands. Qualitative data were collected using semi-structured, open-ended interviews. Qualitative coding techniques were used for data analysis, to identify the challenges of engineering quality requirements, the mechanisms behind the challenges and the practices used that could mitigate the impact of those challenges. Further, by using dialog mapping technique for qualitative data structuring, we have mapped the identified mechanisms and practices to the challenges.

Results: From the perspective of the participating practitioners, our exploratory study revealed 15 challenges classified in five categories: (1) team coordination and communication, (2) quality assurance, (3) quality requirements elicitation, (4) conceptual challenges, and (5) software architecture. The study has also disclosed 13 mechanisms behind the challenges and 9 practices that could mitigate the impact of those challenges.

Conclusions: The main contributions of the paper are: (1) the explication of the challenges from practitioners' perspective and the comparison of our findings with previously published results, (2) the description of the mechanisms behind the challenges, and (3) the identification of the practices currently used by agile teams that could mitigate the impact of the challenges. The findings of this study provide useful input into the process of designing possible solution approaches to overcome the challenges.

1. Introduction

In the past years, much has been published to address the question of whether agile methods are suitable for large-scale distributed projects [28]. Increasingly more evidence through case study research has been provided to suggest that the fundamental assumptions of agile development methods are challenged when applying the methods to large-scale distributed context [27,28]. The present paper focuses on one particular facet of scaling agile, namely the engineering of quality requirements (QRs), such as security, usability, reliability, performance. Our interest in this topic is grounded on a 2017 systematic literature review (SLR) [1] that we carried out on the topic of engineering QRs in agile projects in general. This SLR revealed twelve QR-specific challenges that could lead to a project failure, according to published scientific papers. However, the SLR also found that most literature focuses on the outcome

of the QRs methods and the desired state of the project after the implementation of a particular QR (be it security, or usability, or performance), while the exact aspects that are challenging for engineering QR are not explained in detail and depend on each QR's particular context. Plus, our SLR also reported the lack of empirical evidence on how agile projects handle QRs systematically in their entirety, particularly in agile large-scale distributed context. The results of the SLR encouraged us to initiate an exploratory qualitative interview-based study on understanding the challenges that teams experience in agile large-scale distributed projects when engineering the QRs and the practices that they use to counter their challenges.

We aim at answering the following research questions (RQ):

RQ1: What challenges do agile practitioners face when engineering the QRs in agile large-scale distributed projects?

RQ2: What are the mechanisms behind the reported challenges?

* Corresponding author.

E-mail addresses: w.h.a.alsaqaf@utwente.nl (W. Alsaqaf), m.daneva@utwente.nl (M. Daneva), r.j.wieringa@utwente.nl (R. Wieringa).

RQ3: What are the practices that agile teams currently use to mitigate the impact of the reported challenges?

For the purpose of our research, we consider an agile project ‘distributed’ if it consists of more than one team and its teams are distributed in terms of the distribution models described by Larman and Vodde [2], viz. multi-site teams - the teams work on different locations, but each team is single site, and dispersed teams - the teams work on different locations, and each team is multi-site. Using this definition, we selected and interviewed 17 practitioners employed in different capacities in agile large-scale distributed projects in the Netherlands. The practitioners are drawn from a broad range of business sectors (e.g. banking, public transportation, government service agencies) and areas of professional expertise (e.g. testers, architects, scrum masters, managers, developers). Applying qualitative data analysis techniques from the Constructive Grounded Theory [16], we explicated the challenges in agile large-scale distributed context and their countering practices from the perspectives of those working in the field. The key results of the study are threefold: (1) a catalogue of contextualized descriptions of the practitioners’ views on QRs issues and on these issues’ characteristics; (2) a list of underlying mechanisms responsible for the challenges, and (3) a catalogue of solution practices that the practitioners in our studied projects used to mitigate the effect of the challenges. The findings are compared with those from published relevant studies in the fields of RE and agile software engineering. Implications for research, practice and education are drawn as well. We make the note that first results of the study have been presented at the 2018 international conference on Requirements Engineering – Foundation for Software Quality (REFSQ’18 [38]) and the 2018 international conference on Requirements Engineering (RE’18 [21]). The present paper extends these conference publications by providing new results in the form of new concepts that came out of our detailed data analysis, plus richer and deeper descriptions of those concepts already mentioned in our two previously published papers [21] and [38]. Next, in this paper we provide an in-depth analysis of related work by including empirical studies on agile RE in large-scale distributed context, published until Jan 11, 2019, which we use to evaluate our results in light of published findings of other authors. Third, we provide a detailed methodologically-grounded description of the choices behind our research design. Fourth, we extended discussion in depth and breadth (e.g. by including aspects such as theory-building, realism of generalizability claims among other topics).

This study extends previous empirical RE studies on agile engineering of QRs in several ways. First, we add to the body of empirical RE research on agile large-scale distributed contexts that until now is insufficiently addressed [29]. We explore QRs challenges by taking the perspective of practitioners working in agile large-scale distributed context. In this sense, our case study helps closing an existing gap of knowledge [13–15].

Second, our paper responds to the call of the agile SE community for more empirical research understanding what works and what does not in that context [30]. In the agile SE area, many large-scale frameworks have been proposed (e.g. DAD [37], SAFe [13], LeSS [2], and Scrum of Scrums [14]), however lessons learned from their applications have rarely been published. Furthermore, collecting evidence from an exploratory study on QRs in agile large-scale distributed context contributes to the conversation recently initiated in both Software Architecture and RE communities on the multi-disciplinary nature of QRs. As we will see, our results suggest that QRs might have multiple conceptualizations from the perspective of the agile teams working together in an agile large-scale distributed project. Our empirical study suggests that while some practices that have been put forward in the proposed frameworks (e.g. DAD, SAFe) might be relevant to resolve some of the QRs challenges, others demand re-evaluation of what we know about QRs in agile large-scale distributed context by borrowing theories from other disciplines.

The paper is structured as follows. Section 2 provides background and related work. Section 3 presents our research process, Section 4 reports our results, and Section 5 discusses them. Section 6 is on our study’s limitations. Section 7 is on implications for research, practice and teaching, and Section 8 concludes.

2. Background and related work

There are five systematic literature reviews (SLRs) [1,4,35,36,54] that informed our research. Our 2017 SLR [1] on engineering QRs has indicated the lack of empirical research specifically devoted to the challenges that agile practitioners face when dealing with QRs as a whole in agile large-scale distributed context. The SLR of Dikert et al. [4] on how agile methods have been adopted at scale in large organizations, indicated 7 challenges related to requirements and their testing. These authors’ results point out the testing of QRs as problematic in agile large-scale distributed context. Next, the SLR of Alzoubi et al. [35] on the communication difficulties in “geographically distributed agile development” and the techniques to them, uncovered 17 communication challenges and more than 20 strategies, tools and agile practices to overcome them. Regarding RE, the authors pinpoint to the lack of ongoing interaction between developers and customers and the lack of customer involvement as major challenges. Furthermore, the SLR of Inayat et al. [36] on challenges and solution practices in agile RE indicated QRs as a challenged area and recommended more future research in order to understand the QRs issues in agile. Finally, the 2018 review of Uludag et al. [54] on the challenges specifically in agile large-scale context pointed out to the challenge of making sure that QRs are considered by the development team. Drawing on these SLRs and on our own search for more recent empirical studies (published after the search date of the 2017 SLR [1]), our related work is focused on 15 studies [3–10,18,20,23,27,35,39,40] reporting challenges of engineering requirements in large-scale agile context, which we summarize in Table 1. The first column describes the reported RE challenges and the second – the literature sources. Not all sources mention whether their studied projects were distributed or not. Also, not all indicate the exact size of the case study projects subjected to research. Regarding project size, Käpyaho and Kauppinen [3] classified the reported case study as large where approximately 30 agile practitioners were involved. Dikert et al. [4] classified the case studies they included in their SLR as large. The median size of involved agile practitioners was 300 participants. Kasauli et al. [5] illustrated the size of their case study projects by the number of sub-teams involved (e.g. one project had more than 30 sub-teams, each running its own scrum process). Petersen and Wohlin [6] used the number of involved participants per sub-team and the produced Line of Code (LoC) to denote the size of their case study projects. Each project had an average number of 37 team members and an average of 392,000 of LoC. Rolland [7] described his case study projects as large based on the involved teams and participants. The projects involved 13–14 teams and more than 200 participants. Bjarnason et al. [10] described the company of the studied project as large with around 5000 employees that develop software for the global market. The studied project was considered big as it had to deliver around 1000 detailed requirements. Behutiye et al. [39] investigated the documentation process of QRs in different companies. The size of the involved companies varied between 100 employees for the smallest one and over 100.000 employees for the largest one. The case study did not describe the extent of the studied projects. Dingsøyr et al. [27] reported key challenges of adopting agile methods in large-scale context. The case study investigated a project with twelve co-located agile teams and a total of 175 people involved. The case study did not involve distributed teams and therefore differs from our multiple case study, however the large-scale characteristic of the case study makes it interesting to include as related work. Neto et al. [40] investigated based on a multiple case study the challenges related to RE and software testing in agile context. The authors described their multiple

Table 1

Summary of RE challenges in agile context reported in the literature. Requirements-related challenges are italicized.

ID	RE challenges in agile large-scale distributed context	Source
C1	Minimal documentation and difficulties with getting the teams to document the requirement	[3,5,9,10,23,27,35,39]
C2	<i>The non-availability of customers for requirements negotiation, clarification and feedback</i>	[3,5,7,9,10]
C3	Inappropriate architecture when new requirements arise	[6,9]
C4	Difficulties with requirements-based effort estimation and planning	[4,9,10,20]
C5	<i>Requirements Prioritization based on a single dimension</i>	[6,9,10]
C6	<i>Weak quality assurance practices linked to requirements (e.g. testing of QRs, Automated and integration tests of the requirements)</i>	[3–6,7,9,10,23]
C7	Difficulty to discover dependencies between sub-systems in an early stage	[6,7]
C8	<i>Neglect of QRs</i>	[3,8,9]
C9	Lack of customer's domain knowledge when creating user stories	[10]
C10	<i>Lack of a clear requirements picture early in the development cycle and keeping the focus on the big picture</i>	[3,7,8,10,23,39,40]
C11	Ensuring sufficient competences within the teams	[10]
C12	Difficulties with including innovative ideas from the teams at the time of user story writing,	[10]
C13	Use of prototype code in production, to support requirements validation	[3]
C14	<i>Difficulties with the requirements refinement process</i>	[4,5,23,39]
C15	<i>High-level requirements management largely missing in agile</i>	[4,5,39,23,40]
C16	<i>Difficulties with documenting the QRs in a way that exposes their dependencies</i>	[27,20,23]
C17	Involving all stakeholders throughout the whole agile project	[39,40,18]
C18	<i>Miscommunication of the requirements and having developers base decisions on their experience or guess at the requirements</i>	[18,35]

case study as large, however they did not justify the used of the term 'large' by indicating the size of the studied projects. Holvitie et al. [18] investigated based on a survey the effects of agile development methods on technical debts. The authors got 184 responses from agile practitioners. However, they did not describe the size of the projects the practitioners were involved in. Schön et al. [23], Sachdeva and Chung [8], Ramesh et al. [9] and Mohagheghi et al. [20] did not indicate directly the size of their case studies. As it is hard to understand the relationships between the RE challenges C1–C18 (in Table 1) and the contextual settings in which these challenges were observed, we felt motivated to look closer into the agile large-scale distributed context and do exploratory empirical work focused on QR-related challenges.

3. Our exploratory case study research process

Gaining a deep understand of the challenging situations in the process of engineering QRs in agile large-scale distributed context requires the application of a research methodology that is sensitive to individual perceptions, perspectives and problem-framing and that accounts for the context of social interactions happening in large-scale projects. We designed a qualitative exploratory multiple case study by following the guidelines of Yin [11]. We conducted semi-structured, open-ended, in-depth interviews according to the guidelines of Boyce and Neale [12]. We chose the qualitative interview-based case study approach, because our desired depth of understanding could not be achieved meaningfully through the use of survey questionnaires and closed questions. As per Bensabat et al. [31], employing a case study research method is a particularly suitable to research situations in which researchers study socially constructed processes in systems development projects and seek to achieve as good as possible grasp of reality.

Our research process followed these steps: First, we made a plan describing (1) the kind of information we intended to collect, (2) the kind of practitioners who could provide us with the sought-after information and (3) the kind of project settings that would be an appropriate candidate to be included in the case study. To understand the challenges from different perspectives, we included practitioners with various backgrounds (e.g. different expertise and roles, e.g. architects, testers, different years of experience, different application domains). This is in line with research methodologists [11,12].

Second, the first author developed an interview protocol, which was improved by feedback from senior researchers (the other two authors). We then conducted a pilot interview with an agile practitioner to check the applicability of the questions in real-life context. No changes made to the interview questions after this stage. Interested readers can find our interview questions by clicking the following

link: <https://wasimalsaqaf.files.wordpress.com/2017/07/interview-questions.docx>. The set of interview questions includes two parts. The interview starts with questions to understand the project context, and continues with questions about the practices the participants experienced in engineering the QRs in one particular project of their choice. All interviews were audio-recorded. The audio files were transcribed to a written version by a professional external organization. We chose not to do the transcription ourselves to avoid any interpretation bias that could be passed into the transcripts by the researchers involved in preparing and taking the interviews.

Third, once the interview transcripts were available, two researchers (Alsaqaf and Daneva) conducted qualitative data analysis by drawing upon the analytical practices of Charmaz [16]. To ensure data validity in this research process, a senior peer reviewer (the third author) with expertise in case study research methods completed the research team. Peer reviewer's feedback was integral in ensuring the consistency and accuracy in the data collection and data analysis process which included but was not limited to coding and development of conceptual categories (or themes). The auditor reviewed data to ensure data findings, meanings, and interpretations were supported in the data.

The three steps of our research process are described in more detailed in the next three sub-sections.

3.1. Recruiting participants

The practitioners who joined our exploratory study were drawn from various organizations in the Netherlands. The term 'organization' used in this paper refers to the company that employs the participant and not to the agile project organization to which the participant was temporarily assigned. The organizations included in the case study all claimed to follow agile development methods. We considered one criterion for choosing the organizations: they should use agile approaches to their large and distributed projects. The anonymized information about the organizations is summarized in Table 2. Three of the organizations (O2, O4, O6) have a long history in IT consultancy. They employ high skilled consultants and IT coaches specialized, among other things, in agile development methods. One is a very big government organization (O5) that has adopted an agile large-scale framework for several years. Organizations O1 and O3 provide customized IT services. O1 is specialized in providing transport services and O3 provides administrative software (O3). Both O1 and O3 use an agile development method to develop their software for several years. The second column of Table 2 indicates the approximate size of each organization, the third column shows the organisational domain, the fourth column indicates how many projects from each organization we have included in our study, and the rightmost

Table 2
Case study organizations.

Organization	Size in employee's number	Domain	# of projects	# of participants
O1	Medium (51–200)	IT services provider	2	4
O2	Medium (51–200)	IT and management consulting firm	1	2
O3	Large (200–500)	IT services provider	1	1
O4	Large (300–700)	IT and management consulting firm	3	3
O5	Enterprise (10,000–30,000)	Very big government organization	3	3
O6	Enterprise (50,000–100,000)	IT and management consulting firm	4	4

Table 3
Case study projects.

Project	# members/teams	Scaled-framework	Domain
P1	21/3	None	Public sector
P2	24/2	None	Public sector
P3	117/13	SAFe [13]	Government
P4	30/3	None	Commercial
P5	50/5	Scrum of Scrums [14]	Banking
P6	175/25	SAFe [13]	Commercial navigation
P7	56/7	None	Public sector
P8	12/2	None	Public sector
P9	28/4	None	Government
P10	40/6	None	Health care
P11	27/3	SAFe [13]	Government
P12	24/3	SAFe [13]	Government
P13	14/2	None	Insurance
P14	200/22	Spotify [15]	Telecom

column shows how many participants from each organization joined our study.

After choosing the organizations, the selection of participants from these organizations was based on five criteria: (1) being assigned to at least one large project, (2) having at least three years of work experience in agile, (3) being involved in a RE activity concerning QRs, and (4) acceptance of the invitation to participate in the study. A total of 17 practitioners from the six organizations met these inclusion criteria and agreed to participate in the study.

Table 3 presents the studied projects' settings. All projects used Scrum [19] as their agile development method of choice. One project (P13) fell into the dispersed team category, while the other 13 projects (P1–P12 and P14) were composed of multi-site teams. The second column of Table 3 shows the total number of team members and the number of agile teams in the respective project. For example, project P1 had 21 team members that formed 3 distributed teams. The third column shows which agile scaled framework is used by each project. A cell with 'none' means that no particular framework was used for guidance. The rightmost column indicates the application domain.

Table 4 shows the years of work experience each participant has in general in the field of Software Engineering (SE), and which role(s) (s)he performed in her/his projects which were described in Table 3. Some participants performed more than one role in the respective project, so the number of roles (20) is larger than the number of interviewees (17).

3.2. Executing our data collection plan

The interviews happened between February and April 2017 and were conducted in Dutch by the first author. The length of the interviews varied from 50 to 95 min. At the beginning of each interview, the research objective and the structure of the interview were explained. The researcher informed the participants further about their rights and responsibilities towards the research. Each interviewee gave his/her consent to have the interview digitally recorded and then transcribed, and analysed for research purposes. This allowed us to audio-record all interviews to avoid loss of data.

In the course of the interviews, the interviewing researcher guided each interviewee to focus on a recent project in which the interviewee had experienced challenges with QRs and had to work towards these

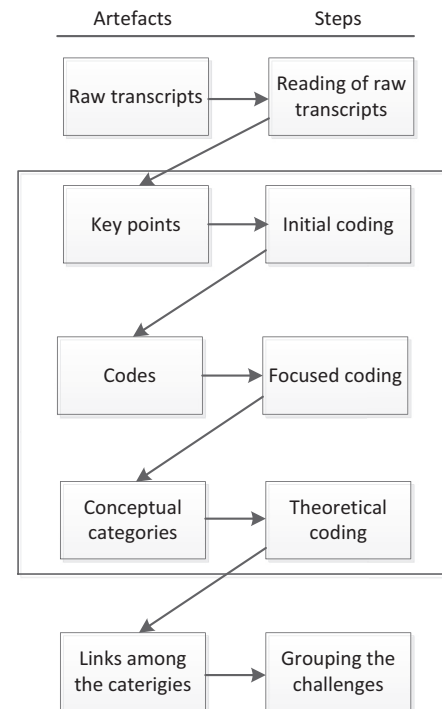


Fig. 1. The coding process in our study (according to Charmaz' guidelines [16]).

challenges' resolutions. The researcher welcomed both successful and less successful examples of project episodes in which the interviewee was involved and had to deal with QRs issues. Each practitioner was encouraged to share specific perceptions, experiences and concerns from his own perspective. In each interview, the interviewing researcher took conscious effort to minimize his interruptions to the minimum and give the interviewed practitioner as much time as necessary to elaborate and explain any particular issue.

Once all interviews were over, the audio recordings were provided to a transcribing services professional company, which delivered the transcripts. The volume of resulting pages of transcribed text was 337 pages. The text was then subjected to data analysis.

3.3. Executing our data analysis process

The focus of our data analysis was to identify and explicate (1) the difficulties, issues and working practices of the practitioners regarding the engineering of QRs, and (2) the elements of context that contributed to generate the emergent challenges. The analysis process in this paper followed the grounded theory method described by Charmaz [16] which is suitable for qualitative exploratory research where theory should emerge from the data. Our analysis process is summarized in Fig. 1. It included three analytic steps: (1) initial coding, (2) focused coding and (3) theoretical coding.

The first step was to break down the interview data into key phrases and to eliminate the irrelevant ones. We attached descriptive labels (called codes) to segments of texts of each transcript. Table 5 provides

Table 4
Years of experience and roles of the participants.

Participant	Years of experience in SE	Project	Role
PA1	4	P1	Software developer
PA2	20	P1	Software developer & Software architect
PA3	15	P2	Scrum master
PA4	36	P2	Software tester
PA5	21	P3	Scrum master & Software tester
PA6	6	P4	Scrum master
PA7	20	P5	Agile coach
PA8	22	P6	Agile coach & Product owner
PA9	10	P7	Software architect
PA10	29	P8	Delivery manager
PA11	25	P9	Software architect
PA12	22	P10	DevOps manager
PA13	17	P11	Scrum master
PA14	15	P12	Software designer
PA15	18	P7	Information analyst
PA16	5	P13	Software developer
PA17	7	P14	Agile coach

Table 5
Texts and codes.

Original text	Codes
PA8: “To avoid any conflicts and useless discussions with the business representative we decided to divide the cake. Hence, we maintained three product backlogs”	Division of requirements documentation
PA4: “We wanted to make the software modular, so we wanted to have the frontend and the backend really well separated. So we said: you know what, we force that by assigning them to two different teams. One team has to implement the frontend and the other the backend. Hard agreements need then to be made regarding the interfaces”	Components’ teams

two examples of the process of coding a segment of text. Next, the focused coding was to organize the codes into conceptual categories. These synthesize and explain larger segments of data [16]. In this step, we searched our interview transcriptions for reoccurring issues, events, and significant factors related to QRs challenges. Finally, the third step was to establish possible relationships between those categories that came out of the focused coding [16]. The possible relationships between categories were explored through creating mindmaps.

The first two researchers (Alsaqaf, Daneva) started by reading the transcripts separately and then inductively applied the step of initial coding. In the focused coding step, these researchers came together and discussed the descriptive codes they applied. Similar descriptive codes were combined in higher-level categories. Different descriptive codes were resolved by conducting an argumentative discussion [17] between the two researchers to reach a shared rationally supported position and then combined in higher-level categories. No unresolved different descriptive codes remained after this step.

4. Findings

Our data analysis process resulted in the identification of 15 challenges. We grouped them in 5 classes, which are presented in Section 4.1. Besides, we distinguished 13 mechanisms as possible causes behind the identified challenges. Those mechanisms are presented in Section 4.2 and mapped to the identified challenges in Section 4.3. In Section 4.4 we reported the practices used by the agile teams in our multiple case study to possibly mitigate the challenges. The practices were mapped to the identified challenges in Section 4.5. As it is usual in qualitative studies, we supplement the findings with interviewees’ quotations. These are given in italic.

4.1. RQ1 - What challenges do agile practitioners face when engineering the QRs in agile large-scale distributed projects?

4.1.1. Teams coordination and communication challenges

These challenges concern the sharing of information resources among the teams within a large Agile project team (e.g. in a scrum of

scrums setting) or among the development teams and the client’s organization. These are:

4.1.1.1. Late detection of QRs infeasibility. In the experience of our participants, inappropriate work coordination and insufficient communication among distributed teams can result in figuring out too late that a needed QR is infeasible. This may cause expensive refactoring of the software architecture and re-implementing the delivered functions. Project P7 was supposed to deliver a web-based system for public use. The system should at all times (1) ensure the authentication and authorization of the user (e.g. security), (2) ensure ease of use by providing the user with all needed information and navigation options (e.g. usability) and (3) ensure high speed by loading the start web-page within 3 s (e.g. performance). The development team discovered at an advanced stage that collecting all the needed information after passing all the security filters will cause the start web-page to take at least 10 s to load. At this point a decision was made to drop the performance requirements since demanding these requirements will result in costly architectural rework. The team traced this issue to the fact that the QR conflicts were not apparent in the beginning and the agile working process had no way to anticipate such conflicts.

4.1.1.2. Hidden assumptions in inter-team collaboration. Our data indicate that QRs are rarely implemented in a single piece of code and can span the whole system. So they typically are the responsibility of different teams. These teams should establish an unambiguous interaction process to ensure the right implementation of the QRs that they share responsibility for. However, often teams make the assumption that every other team responsible for the implementation of a QR understands its part and will implement it correctly, so the resulting system would demonstrate the quality aspect demanded in the QR specification. In the experience of our participants, this assumption however turns out unrealistic. For example, in project P1, text documents had to be made available for end-users to search through. The documents were developed by one team and made available for end-users by another team. This is on the assumption that the documents are correct and accurate. PA1 reports: “We had agreements about, for example, the validity of the doc-

uments. We agreed to put the word “expired” in the name of the document when a document is no longer valid. If the communication between the teams has not gone well – what actually happened- the end-users could consult document which did not reflect the reality at that moment”. Another challenge that our practitioners found was that if a team accepts ownership over the correct implementation of a QR, then this team assumes to rely on the knowledge shared with other teams and also on their availability. Participant PA5 reports the following regarding teams’ interactions: “We want with pleasure to pay more attention to security tests. However, because we are a team among many teams, we have a lot of interactions with those teams. So, our work is still very disturbed by questions from other teams. It’s just awkward. The same with performance tests, we just do not have time to do, or to collect knowledge about those tests. It is just very, yes, frustrating sometimes”.

4.1.1.3. Uneven teams maturity. All our participants indicated that in their perception, the success of agile projects relies on the tacit knowledge embedded in the teams, especially the knowledge related to architecturally significant requirements. They thought, experienced developers are more likely to make better architectural decisions than junior developers. However, when organizations form agile teams, usually these include a mix of experienced and junior developers. In turn, they face the challenge of transferring the knowledge from the more experienced to the less experienced team members in a way that allows both sides to share the same knowledge of the system and enhance the overall quality of it by implementing the right QRs in the right way. Participant PA6 describes his experience: “When we were a small company, we attracted only junior developers. Our senior developers made a strategic decision to build super libraries to deal with issues like security and performance. The novices could use the libraries without changing their internal structure. However, due to misunderstanding of some concepts or using the libraries in a wrong way by novices, performance problems have arisen which resulted in conflicts”.

4.1.1.4. Suboptimal inter-team organization. Large agile projects that include multiple teams, face the challenge of organizing these teams around the so-called Product Backlog Items (PBIs) [19]. Our participants were well aware of multiple ways to organize the agile teams in an agile large-scale distributed context, however they did not know what way would work best in a particular context. They had no reliable way to predict the coordination costs associated with a particular organization of the teams around the PBI. The PBIs are all the desires that might be needed in the product and are listed in an ordered way in the “Product backlog” (PB) [19]. The PB is the single source of requirements for any change to be made to the product. Our participants used various approaches to this situation: (1) Component teams are organized around particular components of the system such as a database, user interface, etc. (2) Feature/Scenario teams are organized around particular product features such as login, log processing, etc. (3) Functional teams are organized around a single development function such as a test team or an architecture team. Depending on the context and the system to be implemented, one of the approaches or a combination of two or more could be used. However, since each of them has advantages as well as drawbacks, our participants thought that teams should be careful with their choice because a suboptimal choice could affect negatively the quality attributes of the system. Participant PA9 describing this challenge: “When we started the project we divided the teams into scenario teams. Each team was responsible for the implementation of a whole scenario from the user interface through the database layer. We saw then that each component suffered from ambiguity and clear guidelines. Because each team had its own way of working and there was no ownership for the components, spaghetti code began to arise and it was difficult to understand the structure of the different components”.

4.1.2. Quality assurance challenges

These challenges concern the activities of verifying and validating the requirements. Our participants describes the challenges they face regarding quality assurance as follows:

4.1.2.1. Inadequate QRs test specification. According to our participants’ observations, QRs are difficult to model and, in turn, identifying and designing acceptance tests for them may be difficult, too. Besides, agile development methods lack formal modelling of detailed requirements [3] which makes the process of verifying the QRs more difficult. PA1 describes this challenge: “At the end of the project an employee was added to the project to perform the acceptance test and he was responsible for data integration and accuracy tests. This tester did not find errors and gave a green light for accuracy, although we know that there are still potential problems”. Participant PA4: “There were no hard performance requirements. So, for example, if I’m doing a test and I feel it takes a very long time, I will give a warning. In principle, it would be good if we had all kinds of requirements on paper or just written somewhere, that is also a kind of a struggle for me, because sometimes I do not know what to test against”.

4.1.2.2. Lack of cost-effective real integration test. All participants perceived integration tests as critical to the verification of the implemented QRs. This was due to the fact that if QRs must be globally implemented, they impact the entire system and not only the components separately. Therefore, the work of the development teams should be merged at some point to perform integration tests. These tests could happen late in the development period. If these tests reveal QR defects, this could result in extremely costly re-work and refactoring of the existing software architecture. For example, P3 was a large project that used SAFe [13] to coordinate the work among the distributed agile teams. P3 had sprints of two weeks and shippable increments every six sprints. At the end of each six sprints, the whole set of all shippable increments delivered by the distributed teams was merged and went through an integration test by a devoted integration team (DIT). The DIT needed other four weeks to complete the needed integration tests. QRs related issues discovered by de DIT went back to the particular teams to be resolved. PA5 reported this challenge: “So what we have done now is actually saving all the work of six sprints and offering it to the integration team at once, while you could actually do the tests in advance”. The development teams do simulate integration test by using mock-ups that represent other teams’ components as part of their own unit tests. However, simulating an integration test is not the same thing as doing a real integration test.

4.1.2.3. Lengthy QRs acceptance checklist. Some of our participants use the so-called ‘definition-of-done’ (DoD) [19] to specify the related QR’s. In agile methods, DoD is the primary check mechanism for agile team members [19] to have the same understanding of when to report a particular task (e.g. user story) as completed. As our participants wanted to include the QRs related to the user stories in development, they had to specify the conditions for deeming the QRs ‘met’ by the delivered product in the DoD. However, specifying QRs in the usual format of the DoD was far from straightforward and resulted in lengthy checklists which impact the development velocity negatively. Moreover, using a long DoD made end-user acceptance criteria, and communication and coordination related to QRs validation more complex. PA8 describes this: “If you have legal or compliance matters, you will at all times comply with the compliance rules of the Dutch Bank or privacy laws, then you have to put them in the DoD, which results in a very long DoD. Every single item in the DoD has impact on the team velocity since you have to do a lot of work to finish one user story”.

4.1.2.4. Sporadic adherence to quality guidelines. Our participants experienced that sticking with quality guidelines was hard to do at all times. They were aware of the need for cross-functional agile teams to coordinate the practices and standardizations in use across all involved teams.

Since the agile teams were self-organized, they had to do the coordination of the used practices and standardizations by themselves. The teams in our multiple case study recognize and acknowledge this matter. PA3: *“For code quality we defined guidelines such as the maximum number of return statements per method, the number of LoC per class and per method, etc.”*. However, we have observed that the distributed teams look at the quality guidelines as something they only have to take into consideration and they do not always have to act according to these quality guidelines. Moreover, the distributed teams do not bother themselves with resolving the violation of the quality guidelines if they think there is no need for that. PA3 explained: *“Sometimes you see that the rule of 20 LoC in a method is violated. This is so trivial, it is self-explanatory. There is no problem about that. We do not stuck to a matrix, it is the just a guideline”*.

4.1.3. QRs elicitation challenges

These challenges refer to different aspects of identifying the right QRs from the right stakeholders. They are as follows:

4.1.3.1. Overlooking sources of QRs. Agile depends on the involvement of the stakeholders to iteratively collect the requirements. However, the scaled agile frameworks deployed in our case study organizations provide no guidance for thorough stakeholder analysis. Indeed, face-to-face feedback sessions were planned to gather stakeholders' feedback on the implemented requirements and to let new stakeholder's requirements emerge. However, to collect those requirements, all stakeholders representing the different viewpoints of the system should be identified. In the experience of our participants, as there was no stakeholder identification process, often agile teams overlooked important stakeholders, leading to missing requirements and, in turn, increasing project cost. As PA7 reflected: *“If I look back at the whole project life cycle, I think identifying all the stakeholders and getting feedback from them as soon as possible is still the biggest threat to the success of the project”*. QRs are by nature cross-cutting requirements which means that they may influence other requirements of different viewpoints. Participant PA9 reports this issue: *“Identifying the QRs was a problem for us. Most QRs were not identified in advance and were discovered in a later stage. By that time it was very complex to implement them”*.

4.1.3.2. Lack of QRs visibility. Our participants were conscious about the categorization of QRs into External and Internal (as in [22]). External QRs are visible to the stakeholders and describe how the system should perform the desired function to be of acceptable quality (e.g. security, performance, availability). The stakeholders of our case study organizations were very interested in those QRs and explicitly talked about them. Internal QRs describe the ease of understanding, maintaining and extending the system (e.g. maintainability, modifiability, extensibility) and contrary to external QRs are in the first instance barely visible to the stakeholders. In the end they are visible to stakeholders, namely by means of increased maintenance cost. Participant PA8: *“Internal QRs get attention only when it is really needed and when the system begins to crack”*.

4.1.3.3. Ambiguous QRs communication process. Our data indicates that establishing a clear QRs communication process is a challenge in distributed agile context that might result in misunderstanding those requirements. The teams were not always able to communicate the needed QRs in unambiguous way. PA5 explained this challenges: *“We used to implement the QRs as they were described in the requirements document which was setup by the PO and the software architect. However, since we started working with SAgE, this document was no longer drafted. Instead we use a feature document which is a description of a particular feature. In the feature document there is a chapter devoted to QRs. However, this chapter contains always only one statement, namely “Not Applicable”. We asked the software architect about this. His answer was: “The general QRs are still applicable and referred us to an old document dated from 2014”. It was absolutely not clear, especially when you have a test manager who specifies which QRs you have to implement in order to comply with the tests”*.

4.1.4. Conceptual challenges of QRs

These challenges refer to the conceptualization of QRs by the agile teams. Specifically, we make a distinction between definitional challenges regarding the nature of QRs, and specification challenges regarding the ways to document them.

4.1.4.1. Unclear conceptual definition of QRs. Participants differed in how they defined the nature of QRs and how they should be treated. Some (e.g. PA8, PA16) experience QRs as stand-alone requirements which should not be treated differently from FRs. Participant PA16 explained: *“Requirements of quality nature as well as those of functional nature can be placed on the Product backlog as well as the Definition of Done”*. Others (e.g. PA11, PA15) do not agree with this statement and emphasized the unique aspects of QRs. They see QRs as constraints on FRs. In their perception, QRs are not separate requirements and always specified in relation to some specific FRs. However, recognizing QRs as part of FRs could result in neglecting the QRs if the related FRs were of low priority and negatively impact the overall quality of the system [9]. For example, A new system with its own data storage process (DSP) was developed to replace the old one (i.e. project P2). However, the new system should operate within the same environment as the old one where the DSM is quite different. To guarantee data consistency and avoid future data errors Participant PA2 proposed to keep the old DSP intact. The product owner (PO) of the project however, did not recognize keeping the old DSP as high priority which resulted in putting the related QR (i.e. data consistency) on low priority.

4.1.4.2. Confusion about QR's specification approaches. Participants indicated that unclear conceptual understanding of QRs brings confusion regarding their recording in the DoD or in the user stories. In those projects that needed to comply with regulations, QRs were elaborated in much detail by using e.g. response time for characterizing performance requirements, but also cyclomatic complexity as a metric characterising the code that implemented the respective QRs (P6). This level of precision was in contrast with “more qualitative” expressions of the other QRs. This variety of specification formats created confusion of how to express which QRs and how to validate them.

4.1.5. Software architecture challenges

These challenges are concerned with linkages between the architecture processes that usually happen in a large project and the agile development processes, and the architect's role and the agile roles in an agile large-scale distributed project

4.1.5.1. Unmanaged architecture changes. Our participants were conscious about the relationship between software architecture and the achievement of. Changes made to QRs at any time in the development cycle could result in costly changes in the software architecture because the earlier architecture becomes inappropriate for the new QRs. Participant PA5 reports this issue: *“We have a number of developers who are already making changes closely to the architecture. Those developers who often have discussions with the software architect whenever he wants to make architectural changes which actually will undermine the overall performance. However, sometimes we choose for more performance and sometimes we do not”*. The many changes in the software architecture could lead to fragmentation of architectural knowledge. The architectural knowledge of a particular system component could be limited to the team responsible for implementing the system component and the overall system architectural knowledge to the role of the software architect. Besides, due to minimal documentation and fragmentation of knowledge, the knowledge about previous architectural decisions can be lost. This could cause the justification of QRs trade-offs already made to be lost, and the software to be less understandable and maintainable.

4.1.5.2. Misunderstanding the architecture drivers. Because agile changes the way in which agile team members interact with the software architect appointed to the large-scale project, our practitioners perceived that there was a lot of room for confusion in the prioritization of QRs for implementation. Our participants gave examples of cases in which there were conflicting ideas of which QRs drive the architecture and why architecture trade-offs are made in a particular way. Usually, in a project one or two QRs are the most important for the architecture and the other QRs are aligned with those. However, in a Scrum-of-scrums, there is no process to assure that teams maintain the same understanding of QRs' priorities from architecture standpoint. Participant PA8 indicated that in a very large project delivering a transportation navigation system, performance and usability were the two most important QRs and all other QRs were consistently checked for their possible effects on these two. PA8 explains: *"It was difficult to define all user experience (UX) items and share them among all distributed teams. Therefore we had a central UX club who monitor and assess all sub-products delivered by the teams based on face-to-face conversations. Besides, we had a dedicated quality tester who monitored the performance of the delivered products using performance testing tools. Any issues reported by those parties propagated directly to the PB"*.

4.2. RQ2 – what are the mechanisms behind the reported challenges?

4.2.1. Implementing QRs based on unstated assumptions

In our multiple case study we observed that agile teams unconsciously make assumptions about the feasibility of QRs, especially when those QRs's implementation depends on re-sources of the customer that the system was built for. For ex-ample, in project P1, agile teams were supposed to implement a highly available system for public use (e.g. 24/7 days). The system depended on collecting its data on a customer's data source which was located behind a proxy firewall. The agile teams did not know about the proxy firewall and assumed the customer's data source would be made available for use. Late in the development cycle, the agile teams discovered that due to other QRs (security) the customer's data source was only available during working hours. The teams were forced then to ad-hoc rearchitect the system by introducing a copy of the customer's data source with higher availability. However, this ad-hoc work-around caused the emergence of data consistency requirements.

Moreover, in multiple agile projects it turned out that new QRs could and did emerge during the development cycle. However, those requirements remained ambiguous until the Product Owner (PO) acknowledged the need for their implementation and hence specified them clearly and unambiguously. In situations where the PO could not specify those QRs, agile teams had to proceed with implementing them based on their judgment. PA2 explains: *"after a while I saw a pattern in how different versions of maps were generated. We could not verify this versioning pattern with the PO, since nor the PO, nor the customer could answer our questions clearly. We decided to implement this versioning's pattern in our software anyway. Fortunately, there was no need to rollback our implementation"*.

4.2.2. Priority assigned to conflicting QRs turns out to be suboptimal

QRs are often related to specific functional requirements (FRs) and rarely act upon the entire set of FRs for a project [41]. In agile context, unrecognized and conflicting QRs can land on the Product Backlog (PB) [19] with different priorities because of their related FRs. The implementation of a QR related to a FR with a higher priority could result in limiting the implementation of a QR related to a FR with a lower priority or eliminating it at all. In project P7 the agile teams had two important QRs among others, namely security and performance. The security requirements were related to authentication and authorization functions which were of high priority. Whereas, the performance requirements (e.g. no longer than three second response time) were related to end-users data retrieval functions with lower priority. The security requirements were implemented by one agile team and resulted in implementing several security filters the data had to go through before being

submitted to the end users. Another agile team had to retrieve the data and made it within three seconds available to the end users. However, by the time the performance related FR got a higher priority, it was not possible to retrieve the data within three seconds. At the end, the performance requirement was dropped since enforcing its implementation would result in unaffordable costs.

4.2.3. Emerging QRs are hard or impossible to implement in the chosen architecture

Architecting software is a solution-related activity, while discovering and specifying the requirements is a problem-related activity [42]. Agile development encourages Just-In-Time (JIT) requirements analysis and implementation which lead to JIT software architecture activities [43]. Therefore, agile teams should be continuously looking for appropriate solutions (e.g. re-architecting the software) when understanding of the problem domain changes (e.g. the emerge of new requirements). We observed that the teams in our study defined and agreed on the overall system architecture early in the development cycle based on limited requirements knowledge. This approach could result in QRs to be infeasible at the time the needs for them emerge. In project P7, agile teams agreed at the beginning of the project on using an event-driven architecture. The choice for this architecture was motivated by the number of existing systems that the new system needs to communicate with. Besides, the teams agreed on making the events as small as possible to avoid network overhead and gain extra performance. In an advanced stage of the project the need for more complex events with more data grew but couldn't be fulfilled due to architectural limitations. As a work-around, the teams queried the database multiple times to retrieve the needed data in sequential small chunks which negatively affected the performance as one of the most important QR's of the project.

4.2.4. Focusing on one's own component and losing the big picture

QRs are by nature cross-cutting requirements. Therefore, the right implementation of QRs could require a well-structured interaction of different system components (Fig. 2). Agile teams responsible for implementing specific components take full ownership of the respective components but may take a careless attitude toward the overall characteristics of the system. PA15 explains: *"We had a software architect responsible for designing the financial component. He overdesigned the system to make it as generic and flexible as possible which resulted in very complex architecture. We could not integrate this component with other parts of the system be-cause we did not understand what goes through it. Therefore, we were forced to re-architecture the component to be able to integrate it"*.

4.2.5. Uninformed choice of an inappropriate communication model impedes the implementation of QRs

The agile philosophy – in contrast to waterfall – encourages interaction over following processes [44]. Therefore, agile teams need to establish well-structured and unambiguous communication channels to ensure the right implementation of the required QRs. The maturity of the involved teams is crucial for establishing and leading their own communication channels. [45] defines two models to coordinate the communication between teams: (1) Coordination between teams is done by a higher authority and (2) Coordination between teams is done by the involved teams themselves. The first option assumes that the teams maturity is not enough to co-ordinate the communication effectively and therefore a higher authority in the form of managers is required. PA6 explained: *"There was no one who banged his fist on the table and said: this is it. There were a lot of opinions, all of which were considered equally important"*. The teams' maturity should be examined up-front to determine the appropriate coordination model. In our multiple case study we did not observe any systematic process used by the agile organization to determine the appropriate coordination model. Choosing sporadically a coordination model that did not match the maturity levels of the teams, in turn, could result in missing QRs. In project P1, text documents had to be made available for end-users to search through. The documents

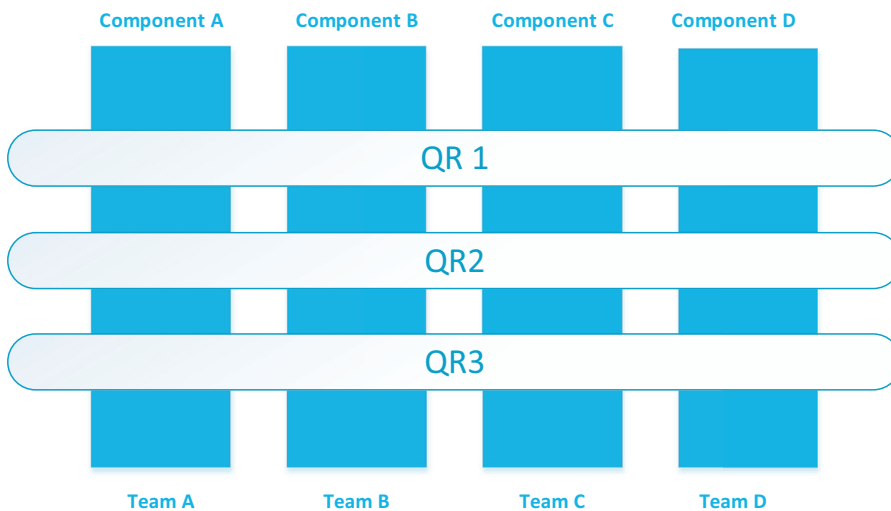


Fig. 2. Cross-cutting QRs.

were developed by one team and made available for end-users by another team. This is on the assumption that the documents were correct and accurate. PA1: “We had agreements about, for example, the validity of the documents. We agreed to put the word “expired” in the name of the document when a document is no longer valid. If the communication between the teams has not gone well – what actually happened- the end-users could consult document which did not reflect the reality at that moment”.

4.2.6. Customers are not interested in internal QRs

When it comes to requirements that are not visible to the end users, the PO as well as the customer are not interested. This moves the focus of the teams from invisible QRs to visible requirements. Besides, the pressure that organizations put on agile teams to deliver functionality together with the lack of interest of the customer in internal QRs gives agile teams a license to neglect the internal QRs. PA2: “It doesn’t interest the PO, the customer at all, how you solve internal technical requirements”. PA14: “To enhance maintainability we had the possibility to switch to a newer version of the currently used framework. However, the customer rejected our suggestion since maintainability had in his perspective a lower priority”. PA8: “What you also saw in agile projects, that when the pressure on the teams is high to deliver functionalities, you saw that the teams remove invisible QRs from the PB to let FR get higher priority. They thought nobody will care about invisible QRs”.

4.2.7. Project-wise thinking prevents the right implementation of QRs

In our multiple case study we observed that those agile organizations that are responsible both for the implementation of the software and for its maintenance, pay much more attention to internal QRs than agile organizations that were only hired to implement the desired system. This mentality was described by PA8 as ‘project company versus product company’, which in fact is translated into keeping a short-term perspective on the project versus a long-term perspective: “What you actually try to do in agile, is to move on from project company to product company. In product company you think more in the future and determine early how you will treat QRs, since every crap you deliver will return back to you again. While project-wise working is more like after me the deluge. When the deadline is approaching, teams have the tendency to compromise QRs in favour of finishing the project in time”.

4.2.8. Not clearly specifying test criteria

To ensure the right implementation of QRs, teams have to verify at some point the existence of the desired QRs. However, to verify the satisfaction of QRs, they need to be unambiguously specified. In agile projects where (conflicting) requirements emerge, agile teams together with the customer fail to specify clearly the scope of the desired QRs. In those cases, verifying the satisfaction of QRs became a matter of feeling

instead of measurement. PA4: “QRs are not specified, neither the happy flow, nor the alternative flows. I try to test both based on my knowledge and experience”. Besides, the impact of conflicting QRs should be examined to specify the acceptable level of satisfaction of each QRs for the customer. By not doing that, agile teams actually leave the testers in the dark. In project P12 where security and capacity were conflicted, PA14 explains: “after delivering the software we got calls from end-users telling us they could not log in. Investigating the log database showed that our security component was not able to handle the login of more than 50.000 users simultaneously. This issue was not tested, since it was not clearly specified. Hereafter we decided to limit the number of users that simultaneously can login to 50.000”.

4.2.9. Focusing on QRs of a specific viewpoint and neglecting those of other viewpoints

Agile teams get their prioritised requirements from the PO. The PO is in the most cases a business representative with particular domain knowledge in his/her area. However, the PO is tasked to speak on behalf of all users from multiple domains, in a project. As the PO may not know all those domains equally well, he/she can pass bias into the prioritization:(s)he could prioritize the QRs of the viewpoint (s)he represents higher than those of other viewpoints. In this way, the PO could put pressure on the team to drop any QRs that (s)he did not request personally, especially in the face of a deadline. Hence, important QRs of different viewpoints could turn out to be in an impasse. P13 had to build a mobile native application for an insurance company. The PO formulated security and usability as the most important QRs. Since mobile devices had limited capacities, the agile teams chose to keep the data alive as a background mobile process for maximum five minutes (e.g. usability) after that a new login should be performed (e.g. security). The PO was satisfied enough with this solution, since the solution met the requirements of the viewpoint (s)he represented. However, users of old mobile devices were less happy since old devices claimed always the memory of applications as soon as they get in the background. Which meant that users of old devices lost their data and had to login repeatedly and more frequently than users of new devices had. Neglecting QR’s of other viewpoints can also happen due to over-looking important stakeholders as indicates by P7: “We had to identify the operational team early in the project as a stakeholder. In our project, the operational team got involved at the end of the project. That was a mistake in my opinion, since a lot of the operational requirements were actually QRs”.

4.2.10. Adopting legacy architectural decisions complicate the implementation of QRs of the new system

Distributed agile teams who are re-implementing a legacy system get advised by the PO to collect important QRs from the existing legacy sys-

tem. From the PO's point of view, since the new system needs to provide among others the same functionalities as the old one, agile teams have to collect the requirements -including QRs- from the existing system. The PO provides them only with new requirements for functionality that does not exist in the legacy system. By lack of documentation, collecting old requirements is sometimes interpreted by the PO as well as by the agile teams as cloning the current architecture and design. Cloning the legacy architecture could complicate the implementation of the new QRs of making them even infeasible. Project P13 was supposed to move a cross-platform mobile application to new platform-dependent (native) application. The old system used cloud storage to enhance security and reusability. The data on the cloud should be synchronised with that on the mobile application based on comparing hash codes. In the new native application, recalculating the hash codes to synchronize the data by every login resulted in a long login process which reduced the usability of the native system. The agile teams were forced to introduce a work-around solution since re-architecting the system was prohibitively expensive.

4.2.11. Moving to agile with a waterfall mind-set

In the waterfall approach, requirements (FRs and QRs) are collected up-front, documented and handed to the software developers to implement them. It is not up to the developers to discover the needed requirements nor to verify them. As Davies [46] describes the behaviour of waterfall developers: *"When requirements come from someone else and someone else tests my work, I don't need to know the environment that my software will be running in"*. On the other hand, agile does not make any differences between the kind of requirements (FRs and QRs) and defines both as user stories (the most used documentation technique as in [47]). Agile developers are expected to collect the needed requirement, elaborate them further in face-to-face communication sessions and implement them just in time. In our participants' projects, we observed that some of the agile developers still had the waterfall mind-set. They implement the user stories that the PO ask for, but do not bother themselves with figuring out what QRs might be needed unless the system begins to expose rigidity. PA1: *"It was not known to the developers that there is a system in between"*. PA8: *"I requested the test manager to give us more days to test the software. I told her the tests we run right now are more waterfall-like while we are doing scrum"*.

4.2.12. Lacking feedback loops on quality standards

Distributed agile teams need to standardize the internal structure of the systems that they are working on, in order to enhance the internal quality and to ensure a shared understanding of the system. We have observed that some of agile teams in our multiple case study develop a kind of nonchalance attitude toward those standards. This nonchalance attitude was traceable to the lack of a "matrix" that measures how far the defined standards have been met and the consequents of not adhering to them. PA3 explained: *"Sometimes you see that the rule of 20 LoC in a method is violated. This is so trivial, it is self-explanatory. There is no problem about that. We do not stuck to a matrix, it is the just a guideline"*.

4.2.13. Ignoring user story priorities at time of pressure

Agile teams prioritize user stories on the PB based on their value to the customer. However, in the perceptions of our case study participants, the priorities associated with the user stories could be shuffled when deadlines are approaching and the need to deliver FR becomes greater than delivering QRs. The commitment to deliver the project in time in fact translates into re-focusing on the FRs alone. As PA8 and PA10 point out: *"When the deadline is approaching, teams have the tendency to compromise QRs in favour of finishing the project in time"* (PA8). *"The hard deadlines we had, forced us to neglect the QRs and focus on delivering the needed functions, despite the fact that the QRs were already defined and described"*. (PA10)

4.3. Mapping mechanisms to challenges

We mapped the identified mechanisms in Section 4.2 to the challenges reported in Section 4.1 by using Conklin's Dialog Mapping technique for qualitative data structuring [48]. It helped us organize the results of our data analysis from each interviewee, by forming challenge-mechanism-arguments structures. The challenges are those reported Section 4.1, the mechanisms are those reported in the previous Section 4.2, and the arguments are the interviewee's facts and reasoning that supported the mechanisms. Table 6 presents our mapping. Therein, the first column shows the categories of challenges as reported in Section 4.1. The second column reports the mechanisms which could result into the challenges of the respective category in the first column. Each mechanism could produce one or more challenges. For example: the lack of interest of the customer regarding QRs (e.g. Customers are not interested in internal QRs) can lead to QRs test specifications and user acceptance challenges (e.g. Quality Assurance challenges). Besides, it also can produce QRs elicitation challenges since the customer does not pay enough attention to internal QRs. Moreover, it can further produce a mix of different QRs specification approaches (e.g. Conceptual challenges of QRs). On the other hand, each challenge can be produced by one or more mechanisms. For example, architectural challenges could be produced by insufficient prioritizing of conflicted QRs, having the agile teams focus on their own components, and losing the big picture of what the whole system needs to be. Furthermore, adopting architectural decisions of legacy system in the new system and making unchangeable architectural choices based on limited knowledge early in the development cycle could also produce architectural challenges.

4.4. RQ3 – what are the practices that agile teams currently use to mitigate the impact of the reported challenges?

4.4.1. Maintaining an assumption wiki-page

In situations where the PO cannot provide clarity about the specification of QRs, agile teams make their own assumptions. These assumptions are also made when the teams think that a QR is self-evident and not worth to be verified. Those assumptions are collected in an accessible wiki-page and implemented together with FRs in different sprints. PA2: *"Half-way through the project we created a wiki-page where we put our assumptions about what might have been QRs. Those were user stories that the PO could not specify enough like how specific the performance should be, whether we have to integrate the current system with other systems already used by the customer, or how to treat versioning. Those were specific technical issues which did not interest the PO"*. The assumed QRs could be rolled back if the PO or the customer did not accept them after implementation.

4.4.2. Use multiple product backlogs to include requirements of different viewpoints

In the experience of our participants, some agile teams explicitly acknowledge that different viewpoints have different interests and QRs. Distributed teams experienced therefore to be useful to have different PBs to document the requirements of different viewpoints. In this way, traceability was assured between each QRs and its respective viewpoint and stakeholders interest. This was important to the agile development teams to escalate possible QRs issues to the respective stakeholder and to allocate those QRs to sprints based on priority negotiation between the respective stakeholders. For example, project P6 had three different PBs. One was the responsibility of the PO and was filled up with user stories that represented the customer's business desires. Another was filled up by the software architect and represented architectural related QRs and the last one was the responsibility of the operation manager and filled up with FR and QRs related to the process of continuous delivery. PA8 explains: *"To avoid any conflicts and useless discussions with the business representative we decided to divide the cake. Hence, we maintained three PBs. The sprint velocity was also divided to reflect the three PBs"*.

Table 6
Mapping challenges to underlying mechanisms.

Challenges	Mechanisms
Teams coordination and communication challenges	-Implementing QRs based on unstated assumptions. -Priority assigned to conflicting QRs turns out to be suboptimal. -Uninformed choice of an inappropriate communication model impedes the implementation of QRs. -Focusing on one's own component and losing the big picture.
Quality assurance challenges	-Customers are not interested in internal QRs. -Not clearly specifying test criteria. -Focusing on QRs of specific viewpoint and neglect those of other viewpoints. -Moving to agile with a waterfall mind-set. -Lacking feedback loops on quality standards.
QRs elicitation challenges	-Ignoring user story priorities at time of pressure. -Customers are not interested in internal QRs. -Focusing on QRs of specific viewpoint and neglect those of other viewpoints. -Project-wise thinking prevents the right implementation of QRs. -Moving to agile with a waterfall mind-set.
Conceptual challenges of QRs	-Implementing QRs based on unstated assumptions. -Uninformed choice of an inappropriate communication model impedes the implementation of QRs. -Customers are not interested in internal QRs. -Focusing on QRs of specific viewpoint and neglect those of other viewpoints. -Moving to agile with a waterfall mind-set.
Architecture challenges	-Priority assigned to conflicting QRs turns out to be suboptimal. -Focusing on one's own component and losing the big picture. -Emerging QRs are hard or impossible to implement in the chosen architecture. -Adopting legacy architectural decisions complicate the implementation of QRs of the new system.

4.4.3. Use automated monitoring tools

Agile teams make use of tooling (e.g. SONAR¹) to monitor the quality of their software under development. QRs that are related to the internal operation of the software were implemented as rules in the monitor tool. The tool then raises a warning when the defined rules are overridden. PA11: *“We tried to automate the Definitions Of Done (DoDs) [19] as much as possible and implement them in Sonar. For example we agreed to use where possible in our code the java keyword Final. It is a small technical detail that can improve the performance. Hence, we implement a Sonar rule that checks de code and gives a sign if it finds no Final in a place where it should”*. PA9: *“We used Sonar as quality application. Code quality rules were implemented in Sonar and any violations of those rules was reported by Sonar”*.

4.4.4. Reserve part of the sprint for important QRs

QRs which in the opinion of the PO do not have business value could turn out to be neglected more easily than others. The neglect of those QRs may result in inflexible and difficult to maintain system. In project P1, agile teams found a way to work around this PO behaviour. They collected the internal QRs in a different PB and agreed with the PO to implement one of those QRs each sprint as long as time permits. PA2: *“We said to the PO, you have to give the teams the space to implement one technical QRs each sprint. The teams did use this space to improve the quality of the software. The PO doesn't even know which QRs we were implementing”*. Other agile teams reserved part of the sprint velocity to be dedicated to important QRs. PA14: *“We have reserved 30% of each sprint for important QRs, such as Maintainability, Performance and Security”*.

4.4.5. Sprint allocation based on multiple PBs

Distributed agile teams could use multiple PBs to include QRs of different viewpoints. Hence, they divide the sprint according to the number and importance of the different PBs. Project P6 had three PBs representing the business requirements, architectural QRs and continuous delivery requirements. The sprint capacity was distributed among the different PBs, respectively (40%, 40% and 20%). For example if an agile team has a sprint velocity of 20 story points then the sprint will have 8 story points of business requirements, 8 story points of architectural related QRs and 4 story points of continuous delivery requirements. PA8 explains: *“40% of the sprint velocity was devoted to business user stories,*

40% was dedicated to architectural QRs and 20% for the pipeline requirements”.

4.4.6. Establishing preparation team

A preparation team is a team that consists of senior information analysts, senior software architecture and business representatives. This team works in parallel to the other distributed agile teams and is responsible for drafting the PB, making the PB items ready for implementation, defining the overall software architecture (e.g. big design up-front [49]) and assigning ready items from the PB to the distributed agile teams. The preparation team begins from a so-called ‘sprint zero’ with gathering the most important requirements and defining the overall architecture. The team continues with collecting the needed requirements, making them ready to implement and refining the software architecture during all the sprints after. The team distributes the defined user stories among the distributed teams based on the nature of the user stories and the available skillsets within the different teams. PA9 explains: *“We had specialized teams. Every team was responsible for a specific component (e.g. user screens team, end-user's letters team). Besides, we worked with scenarios that touch every component of the system”*. The preparation team made the user stories outflow from the scenarios ready to implement and assigned them to the right team with the needed skills. PA12 used in project P10 the same approach, however, they called it ‘Readiness Team’ instead of ‘preparation team’.

4.4.7. Establishing components teams

As we already indicated, the agile large-scale distributed projects in our multiple case study organize distributed teams around particular components. In our participants' experience, component's teams develop an ownership feeling about the components that they are responsible for. This feeling enhances the internal quality of the individual component and hence the quality of the whole system. PA9: *“At the beginning of the project we as-signed a complete scenario to each team to implement. We saw that each component suffered from ambiguity and poor internal quality, since each component could be modified by each team. Then we decided to assign components to teams and make each team owner of a particular component. The internal quality of the components significantly improved”*.

4.4.8. Establishing QR specialists' teams

We have observed that agile large-scale distributed projects dedicate the owner-ship of important QRs to teams with solid knowledge about

¹ <https://www.sonarqube.org/>

Table 7
Mapping practices to challenges.

Categories of challenges	Practices
Teams coordination and communication challenges	-Maintain an assumption wiki-page. -Establish preparation team.
Quality Assurance challenges	-Use automated monitoring tools. -Establish QR specialists' teams.
QRs elicitation challenges	-Innovation and Planning Iteration (IP). -Establish components teams. -Establish preparation team.
Conceptual challenges of QRs	-Reserve part of the sprint for important QRs. -Use automated monitoring tools -Use multiple product backlogs to include requirements of different viewpoints. -Maintain an assumption wiki-page.
Architecture challenges	-Sprint allocation based on multiple PBs. -Establish preparation team. -Establish QR specialists' teams. -Innovation and Planning Iteration (IP).

that particular QR. For example, if security is an important QR for the project, a team with security specialists will be put together and will be assigned the ownership of security requirements. This team should ensure the implementation of the security requirements across the distributed teams. PA13: *“Performance and usability were dedicated to other teams within the organization. security was dedicated to an external team”*. Moreover, in cases that the system fails to meet the customer expectation regarding a particular QR, a team with solid expertise in that particular QR could be set up in an ad-hoc manner to resolve the failure. PA5: *“The previous version of the project did not meet the performance expectation of the customer. We put together a performance team who should analyse and resolve the problem”*.

4.4.9. Innovation and planning iteration (IP)

IP is a term that is used in SAFe [13]. It is a time period equal to one sprint that an agile team can request in order to work on activities other than delivering user stories with business value. In our study, we have observed that distributed teams request IP to resolve technical debts introduced in previous sprints. PA14 explains: *“Important QRs which had business values (e.g. security, performance) were implemented, other QRs were neglected. We had to fight for an IP to resolve technical debts”*.

4.5. Mapping practices to challenges

We mapped each of the reported practices in Section 4.4 to the reported challenges in our previous Section 4.1. For this purpose, we used the data structuring technique [48] mentioned in Section 4.3. Table 7 summarizes this mapping. The first column of the table represents the reported categories of the challenges. The second column shows the currently used agile practices in distributed context.

Each of the reported practices in Table 7 could (partially) mitigate the impact of one or more of the reported challenges. For example, the practice of establishing a preparation team is used to solve coordination issues. It helps coordinating the collaboration between the distributed teams by eliminating any ambiguity that could originated by team's miscommunication. Moreover, establishing a preparation team is used to control architectural changes and to prevent unmanaged architectural changes from being happened. Besides, the impact of the reported challenges could be mitigated by implementing one or more practices. For example: the challenges of the category “QRs elicitation challenges” can be mitigated by implementing different practice namely (1) Establishing component's teams to enhance the internal quality of the different components, (2) Establishing a preparation team to collect the requirements and release the PO from being the only source of the requirements, and (3) Reserve part of the sprint to be used for implementing important QRs.

5. Discussion

This section puts our finding in the perspectives addressed in prior publications in RE for agile large-scale distributed context. Our discussion follows the topics in the previous sections. The present discussion builds upon the preliminary discussion points that were described in [38] and [21]. It extends these points by adding the discussion ideas that crystalized at both the 2018 REFSQ conference (<http://regsq.org>) and RE2018 conference (<http://re18.org>) when the preliminary results of this study were first presented. The present discussion also includes new points that came out of a reflection that happened as part of preparing this paper.

5.1. Reflection on the findings

We found that practitioners perceived QRs as architectural requirements. They were concerned about the belated clarification of QRs in their projects and wanted QRs to be identified as early as possible and analysed for their architectural impact. This could be considered as a signal to clarify as early as possible the role of architecture in agile large-scale distributed projects and its driving requirements. An open question remains regarding whether it makes sense in an agile large-scale distributed project to define architectural requirements (e.g. QRs) up front. Our results make us think that this makes sense since the emergence of those requirements in advanced stage results in re-architecting the software, leading to avoidable loss of time and resources.

Our consistent observation in the six organization was that agile practitioners did not relate their user stories to QRs and to the requirements in general (be it FRs or QRs). Is the term ‘requirements’ that we know from RE textbooks (i.e. [26]) applicable to agile projects? Or should we (researchers) forget this term and start talking about ‘agile customer desires’? As our agile practitioners did not relate requirements to user stories, they also did not relate the properties of good requirements (that we know from textbooks) to the user stories. When we asked our agile practitioners to tell us about their understanding of the term ‘requirements’ in agile, their collective experience was that in agile projects, the requirements are equal to “the user stories + the conversation about what is in the user stories + the acceptance criteria”.

Our findings indicated a link between QR challenges and the way in which practitioners learn in agile. Agile development is an ongoing learning process that involves tacit knowledge. Senior developers build a solid theoretical and practical knowledge that enable them to find their way easily in a dynamic environment and help them with making right decisions. Junior developers on the other hand do not have that tacit knowledge. The only knowledge they have is what they got at their educational institutes which does not include improvising in dynamic environments. Do we expect junior developers to make right decisions in dynamic environments based on their knowledge collected form

educational institutes? If not, how to ensure they include QRs (which they get at universities) in dynamic environments (agile) which does not recognize terms as requirements.

We have observed that though currently used agile practices could mitigate the challenges, they could also introduce other challenges. In our multiple case study, the agile large-scale distributed projects with multiple distributed teams organize the teams around defined subsystems or components (e.g. payment component, registration component). This approach to the organization of the work was perceived as “something new” in the organizational settings where our participants were employed. In the experiences of our participants, when a particular team has the ownership over a particular component, the team also takes actions to guard the quality of its own component. This is because the technical debt detected in a particular component will eventually return back to the responsible team. Plus, each component will show stability and clarity since only one team is allowed to touch the code belonging to that particular component. Therefore components’ teams [50] could be an effective mitigation against poor internal quality and ambiguity. However, the customer who requested the system, is not interested in the individual components but in the business values delivered by the whole system (made up of these components). If the delivered business value fails to meet the expectation of the customer, the component’s teams would not take the responsibility of the failure since each of them is only responsible for a particular component (and no one is responsible for the system as a whole). Moreover, the component’s teams need together to agree on well-defined communication protocols to ensure the delivery of the desired business values (e.g. inter-faces). In our multiple case study we have observed that teams are not always capable to steer the communication between component’s teams in the right direction especially if the teams suffer from lack of maturity. In case of lack of maturity within component’s teams, a higher management role is needed to coordinate the collaboration between the teams [45]. Feature’s teams [50] is another approach to organize distributed agile teams. The idea behind the feature’s teams is to organize the teams around requested business values (e.g. system features). A feature team is allowed to touch all existing components to ensure the right implementation of the required feature. Feature’s team could be the answer to the challenges of using component’s teams [50]. However, according to our participants feature’s teams lack the responsibility over individual components which could result in delivering subsystems of poor internal quality. PA9 explains: *“When we started the project we divided the teams into scenario teams. Each team was responsible for the implementation of a whole scenario from the user interface through the database layer. We saw then that each component suffered from ambiguity and unclear guidelines. Because each team had its own way of working and there was no ownership for the components, spaghetti code began to arise and it was difficult to understand the structure of the different components”*.

We have also observed that agile teams, specially less mature ones, still need a senior management role to coordinate the collaboration between the distributed teams which is totally against the spirit of agility [19,44]. The agile thinking rests on the self-organizing team concept which implies that a team is capable to organize the needed activities itself to perform the required work. In our multiple case study we found that agile teams use practices that move the elements of self-organizing to a senior management team (e.g. preparation team, ready team). This results in agile teams that are not self-organized anymore. Establishing a preparation team mitigates some challenges, e.g. unmanaged architectural changes, coordinating the communication between the distributed teams and improve the requirements elicitation and specification processes. However, establishing a preparation team takes the flexibility of agile a step backward toward waterfall since the distributed agile teams are stripped out from their self-organizing elements. Besides, the use of a preparation team could introduce other challenges such as making definitive architectural choices early in the process that prevent emerged QRs from being implemented. P7 explains this challenge: *“We made at the beginning of the project an architectural choice. This choice in-*

clude the use of small messages in an even-driven architecture to enhance the performance. I think it was not a very good choice. However, now we cannot change it anymore and we just have to work with it”.

An interesting observation is that agile teams use different approaches to document QRs (e.g. assumption’s wiki, multiple PBs, rules in monitoring tools, etc.). This observation exposes the struggle that agile teams experience when it comes to QR’s specification and documentation. It also shows that agile teams find creative solutions to cope with the inability of user stories to document QRs [52,53].

Furthermore, we wanted to investigate whether the challenges could be associated with specific business domain (e.g. banking, transport) and/or specific type of system (e.g. information system, embedded system). In Table 8, we relate the identified challenges to the contexts in which our participants experienced them. The first column indicated the ID of the challenge, where FC is an acronym standing for “*Found Challenge in this case study*”. However, this analysis did not provide us with enough information to hypothesize that certain challenges could belong uniquely to certain contexts, nor that they could be experienced in all possible contexts. More research is needed in order to understand if there are QR challenges attributable to certain domains. This is treated in more detail in Section 6 where we discuss limitations and validity threats.

Besides, we reflected on the results regarding the extent to which they are specific to agile only. The goal of this exploratory study was to understand RE challenges, underlying mechanisms, and solution practices in agile large-scale distributed context. However, as we can see, some of the challenges that we identified are also common in traditional approaches in large projects. Specifically, the late detection of QRs infeasibility, the challenges of QRs being implemented across components, inadequate QRs test specification, overlooking sources of QRs, lack of QR visibility and unmanaged architecture challenge, assumptions about inter-team collaboration, and suboptimal inter-team organization are common for both agile and non-agile contexts. This possibly means that our findings on how to address these challenges may also help improve software projects that use non-agile approaches. For example, the practices of establishing a preparation team and establishing a QR specialists’ team could be used as a solution to the unmanaged architecture challenge also in non-agile projects. In the same vein, teams in agile large-scale distributed context organize their staff around PBIs similarly to non-agile teams organized around sub-projects each with its own portion of requirements. The Sprint Goal, similarly to the project phase goal in non-agile, embeds a common idea behind the various PBIs and creates the foundation for teamwork. The Product Backlog, in turn, ultimately serves as a master plan for the agile team, just like the “traditional project plan”. We, therefore think that non-agile teams could potentially benefit from the solution practices that we found regarding this challenge. However, our list of challenges has many items that are peculiar to agile large-scale distributed context. For example, the challenge of uneven team maturity is traceable to the way knowledge transfer across team members happens in agile. In non-agile large scale projects (e.g. projects that use gates and milestones in their lifecycles), there are usually predefined procedures for knowledge sharing and communication. Reliance on process and explicitness of procedures offsets the downside that comes with bringing new team members on board. This would not be the case in agile teams that rely on tacit knowledge and face-to-face communication. In the same way, we have observed that some of the mechanisms are not specific to agile large-scale distributed context, but are known from waterfall projects, e.g. that customers have less interest in internal QRs, which in turn create QRs elicitation challenges (see the third row in Table 6). However, the solution practices that agile teams come up with in order to counter these challenges are different compared to those in waterfall projects. While waterfall projects may resort to increasing the number of documentation (e.g. using standards, checklists and explicitly defined QRs-specific terminology [51]), agile large-scale distributed projects put emphasis on people and propose solution practices that are

Table 8
Statements on QRs challenges traceable to contexts.

ID	Challenge and project case in which it was experienced	Business sector	Type of system (information system/embedded software)
FC.1	Late detection of QRs infeasibility	Government	Information system
FC.2	Hidden assumptions in inter-team collaboration	Transport (semi-government) public sector	Information system
FC.3	Uneven teams maturity	Tax services	Information system
FC.4	Suboptimal inter-team organization	Commercial/business	Information system
FC.5	Inadequate QRs test specification	Government	Information system
FC.6	Lack of cost-effective real integration test	Tax services	Information system
FC.7	Lengthy end user acceptance of QRs	Tax services	Information system
FC.8	Sporadic adherence to quality guidelines	Transport navigation	Embedded system
FC.9	Overlooking sources of QRs	Transport (semi-government) public sector	Information system
FC.10	Lack of QRs visibility.	Banking	Information system
FC.11	Ambiguous QRs communication process	Transport navigation	Embedded system
FC.12	Unclear conceptual definition of QRs.	Tax services	Information system
FC.13	Confusion about QR's specification approaches	Insurance	Information system
FC.14	Unmanaged architecture changes	Transport navigation	Embedded system
FC.15	Misunderstanding the architecture	Tax services	Information system
		Transport navigation	Embedded system

organizational in nature – e.g. establish component teams, and preparation teams.

Finally, we asked ourselves if the challenges vary in their severity and possible risks to which they expose the project organization and the company where the project happens. One might assume that some challenges (e.g. FC.13, in Table 8, Confusion about QR's specification approaches) are less severe than others (e.g. FC.14 and FC.15). This means that some challenges could be more an annoyance than an existence-threatening forces, e.g. that could possibly bring an organization to a financial bankruptcy. We must note that none of our case study organizations experienced existence-threatening consequences, such as bankruptcy or a loss of a client. However, all practitioners talked about project delays, and extensions of project duration. We note however that in a descriptive case study like ours, it was difficult to judge if a particular project outcome is causally related to a particular challenge to QRs, as there are so many variables in the project context that might influence positively or negatively the specific project outcome. We therefore think that more research is needed in order to understand and quantify the risk that each challenge could pose to an organization. We consider this research important and plan follow-up case studies, possibly with researchers from other countries, in order to understand how the challenges work in regard to various contextual settings (e.g. organizational cultures, business models, regional mentality and attitudes).

5.2. Comparison with previously published challenges

We now compare our findings with the twelve challenges reported in our 2017 SLR [1]. Our 2017 SLR and the current study agree that agile development methods do not provide any widely accepted technique for gathering QRs. Our present results also agree with the SLR findings in that the identification of stakeholders and their QRs at the right development stage is a challenge. Besides, agile practitioners lack agreement on the nature of QRs and this makes the process of specifying the QRs unclear. However, more research needs to be done to investigate whether the lack of documenting the QRs is due to the inability of the user stories to document them (as reported in our SLR) or due to the lack of agreement about the nature of QRs (as reported above). We use the identifiers FC.1-FC.15 to refer to the challenges from Table 8. Our results also overlap with other previously reported challenges [1], namely, Inadequate QRs verification (FC.5), lack of cost-effective real integration test (FC.6), and unmanaged architecture changes (FC.14). However, we did not find any evidence that the challenges related to the product owner's role, e.g. issues, such as product owner's heavy workload and his/her insufficient availability [1], in no way threatened the success of the projects in our case study.

We also compared our findings with the 18 challenges from literature reported in Table 1, Section 2. We use the identifiers C1-C18 to refer to the challenges from Table 1. Our findings overlap some, but not all of the challenges in Table 1: (C1) *Minimal documentation and difficulties with getting the teams to document the requirement*. Our participants did not mention minimal documentation as a challenge by itself. However, they explained that minimal documentation could cause those architectural decisions and the justification of QRs trade-offs that were already made, to get lost, which in turn will make any further changes to the software a challenging task. (C3) *Inappropriate architecture when new requirements arise*. Our participants acknowledge that the emergence of QRs in a later stage of development makes the adjustment of the software architecture complex and could result in architectural rework. (C6) *Weak quality assurance practices*. Our findings reveal that distributed agile projects struggle with quality assurance practices (e.g. Inadequate QRs verification, Integration tests), which could result in discovery of defects late in the software lifecycle. (C7) *Difficulty to discover dependencies between sub-systems in an early stage*. Our results did not mention this as a challenge. However, our study revealed that practitioners could overlook the consequences of such dependencies which could result in not delivering the demanded QRs. (C8) *Neglect of QRs*. The participants in our case study recognize these challenges (see Sections 4.2 and 4.3). Due to complexity of identifying and verifying QRs at an early stage of the development cycle, a fast and non-elegant implementation of the QRs could be the result later in the development cycle. (C10) *Lack of a clear requirements picture early in the development cycle*. Our practitioners mentioned the late detection of QRs as a challenge for implementing them correctly (see Section 4.1: 1.1). (C11) *Ensuring sufficient competence within the teams*. Our findings suggest that agile teams face the challenge of transferring relevant knowledge from experienced team members to less experienced members, which could result in an insufficient distribution of competences among the distributed teams. (C16) *Difficulties with documenting the QRs in a way that exposes their dependencies*. The practitioners in our multiple case study acknowledge the importance of understanding QRs dependency, however, they do not recognize it as a challenge. According to our participants, the way to explore and examine QRs dependencies is to implement the requirements and publish the software to the customers to collect their feedback as soon as possible. Only then, in the experience of the practitioners, one will find out whether the needed QRs are implemented and whether they impact each other in any way. Despite the observation that our practitioners do not recognize QRs dependencies as a challenge, our data shows that an inappropriate identification of conflicting QRs could result in the infeasibility of needed QRs. (C17) *Involving all stakeholders throughout the whole agile project*. Our data did not mention this as a challenge, however, our

practitioners had difficulties with identifying the right stakeholders to collect their requirements. Overlooking stakeholders is one of the challenges our practitioners cope with. Once the stakeholders are identified, including them throughout the whole project is not a problem from the perspective of our practitioners. (C18) *Miscommunication of the requirements and having developers base decisions on their experience or guess at the requirements*. Our findings indicated that establishing unambiguous communication channels between the distributed agile teams is a challenge which can result in misunderstanding the QRs (see Section 4.1: 3.3). Besides, we have found that due to misunderstanding the QRs agile teams could make unverified decision regarding the implementation of QRs (see Section 4.1: 1.2).

Our findings did not confirm the following challenges reported in Table 1: (C2) The non-availability of customers for requirements negotiation, clarification and feedback, (C4) Difficulties with effort estimation and planning, (C5) Requirements Prioritization based on a single dimension (C9) Lack of customer's domain knowledge, (C12) Difficulties with including innovative ideas from the teams, (C13) Use of prototype code in production, (C14) Difficulties with the requirements refinement process and (C15) High-level requirements management largely missing in agile.

6. Limitations

We evaluated the threats to validity by using the checklist of Yin [11]. The threats concern generalizability, reliability and construct validity. Because our research does not seek to establish any causal relationships, we do not discuss threats to internal validity [11].

6.1. Generalizability

Would our findings hold if we would interview other practitioners in other countries? We can only claim that the reported challenges occurred in the projects reported above by our subjects. In addition, ten challenges identified by us have also been identified by other researchers (C1, C3, C6, C7, C8, C10, C11, C16, C17 and C18). We think that the desired generalization is not that the challenges we identified occur in all agile large-scale distributed projects, but that they may occur more often in this context, and are important to understand, prevent and mitigate. Our research provides evidence that these challenges are important, because practitioners reported them as such; whether they occur more often requires more research.

Moreover, one might assume that some QRs might be characteristic to a certain business domain (as in [33]) and, in turn, some challenges could possibly be specific for this domain. Our data (Table 8) does not allow us to conclude if this is the case or not. In our exploratory research, we were striving for breadth and not for depth. More empirical research is needed in various domains in order to understand the QRs challenges specific for each domain, and those that are observable across domains. This can be investigated by a comparative case study of two case study organizations that use the same framework for scaled agile (e.g. SAFe) but operate in two different business sectors. As we are interested in this topic, we are planning this to be our next step.

6.2. Reliability

It might be possible that the researchers passed occupational bias [24] into the research process due to their own business experience (that they accumulated prior to their academic careers). Specifically, the first author has an agile software engineering background, therefore, some occupational bias [24] could be passed to the interview questions as well as the interviews themselves. This type of bias was reduced by (1) having the interview protocol and questions reviewed by experienced and senior researchers (the second and third authors); (2) conducting a pilot interview to ensure the applicability of the interview questions; (3) recording all the interviews and having the audio files reviewed by the

senior researchers; and (4) having the audio files transcribed to a written version by a professional external organization. Following Adler [24], we considered that the purpose of our bias-reducing efforts was not to strip out the insider's status of the two authors regarding the interviewees (sharing the characteristic, role, or agile experiences under study with the participants). Instead, the purpose was to make sure that our interview questions were thoughtfully posed and delivered in a way that allows our interviewees to reveal their true feelings without distortions. In line with this, we treated with special care the minimization of confirmation bias, specifically, the first two authors continually re-evaluated what we read in the transcripts and had the third researcher challenges any pre-existing assumptions and hypotheses.

Another reliability threat is our participants' bias due to a possible lack of honesty [25]. We countered this limitation by taking three actions: (1) we assured that all the participants were volunteers and had the right to refuse answering any question at any time or even leave the interview at any stage without giving a reason; (2) we assured the participants that all information remains confidential and anonymous; (3) the interviewer started each interview by explaining the objective of the research to the participants and the importance of giving accurate and honest answers to the validity and reliability of the research. A third possible weakness of interview techniques is the tendency for the interviewer to ask leading questions [25]. However, this threat is minimal since we conducted a pilot interview to ensure the applicability of the interview questions after having the interview questions reviewed by the senior researchers.

6.3. Construct validity

We used four measures to mitigate construct validity threats: (1) triangulation of "multiple sources of evidence" (as recommended by Yin [11]), (2) member-checking [12], e.g., during the interviews, the interviewing researcher restated or summarized the information provided by the interviewee and then used follow-up questions to the respective practitioner in order to determine accuracy; (3) peer debriefing [32], which happened through the involvement of senior researchers, and (4) disclosure of researcher bias [32], which was achieved by maintaining a reflective journal. After every interview, the interviewing researcher spent time noting his immediate observations, thoughts and interpretations before he subjected the data to coding.

7. Implications

Our findings carry important implications for researchers, for practitioner, and for teachers in Requirements Engineering, Agile Software Engineering and other related fields included in the Computer Science education.

7.1. Implications for RE research

Our study found that the QRs are more than what can be included in user stories. In fact, the user stories and the conversations following up on them, are forming together the QRs in a project. Who exactly shapes these conversations and what their contents are, however needs further research. For example, one might assume that in agile large-scale distributed projects, the PO and the architect might both share the responsibilities and the leading role concerning the engineering of QRs. Whether this is the case, could be determined only after empirical studies in industry. The practitioners in our multiple case study recognize the importance of analysing requirements dependency and making the requirements traceable. However, due to the nature of agile where requirements continuously emerge and are not collected upfront, analysing requirements dependency could not be done in a waterfall manner. More research needs to be done to explore the way agile practitioners handle requirements dependency.

Furthermore, we acknowledge that there might be possible differences between the challenges that organizations experienced, depending on the type of each organization (e.g. government agencies versus private companies). For example, one could imagine that the structure of the governmental organization has a significant impact on the way a project works and, hence, may also have an impact on the QRs challenges. Understanding such possible relationships is an interesting industry-relevant topic and we consider it a worthwhile line for future research. Finally, it is very interesting to understand what the specific risks are that our challenges expose organizations to. Are there any causal relationships between a specific challenge and its mechanism and a specific project outcome? We consider this research very important and call for more empirical studies in this area.

7.2. Implications for practitioners

This study has some implications to RE practice. Perhaps the most important finding that revises our understanding of agile is the role of the user story as a requirements artefact. Our participants indicated that user stories are not and cannot be a replacement for requirements. In our study, user stories were perceived as commitments to a conversation about the QRs, and the total requirements were in fact the sum of the user stories plus the contents of the conversations. This has implications for POs and junior agile developers. If user stories need to be complemented with conversations for the purpose of QRs, then for POs this would mean that they might want to consider becoming more proactive and make sure that conversations on QRs happen early enough in the project and not later when some architecture design choices have already been made. Next, to junior developers, this finding means that they should learn to anticipate conversations on QRs following up the user stories that they work on, and to treat these conversations on QRs seriously and acknowledge their status as part of the requirements.

Next, our findings suggest that practitioners should be aware of the presence of tacit assumptions surrounding the QRs in agile large-scale distributed context. While the FRs get explicit in the form of user stories, the QRs silently exist and remain undocumented for an extended period of time which might be detrimental for the project.

Our study also indicates that once the QRs are identified, practitioners do not share a common perspective of documenting those requirements. The study shows that practitioners use different approaches to document the QRs (e.g. some put them as user stories in the product backlog, while others embed the QRs specifications in the definition of done). This variety of documentation strategies leads to introducing inconsistencies in the development process, which in turn, could result in nonchalant behaviour regarding the documentation of QRs. More research is needed to evaluate whether a general perspective regarding QRs documentation is necessary and possible approaches to probably document the QRs.

7.3. Implications for RE education

This study has some implications for RE educators. First, agile software engineering textbooks convey that user stories are enough to document the requirements. Our study indicates that this might not be the case. In fact, our participants were united on the matter that a user story in an agile project is not a requirement, but a *commitment to a conversation on the QRs*. RE teachers should consider that agile projects would call for RE skills that are currently not covered in well-known agile textbooks (e.g. [27]) and therefore should search for ways to educate students on QRs in agile large-scale distributed context and help them get exposure to this specific context.

Second, our study indicates the QRs are shared responsibility of software architects and POs. That architects are actively involved in engineering QRs has been discussed in a number of empirical studies (e.g. in contract-based projects [33,35], in near-shore development context [34]) which are not agile large-scale distributed context exclusively.

This means, very large project regardless if they are agile or not, would demand a RE specialist to collaborate with a software architect for the purpose of QRs engineering. To RE teachers, this means that teaching agile RE should be aligned not only with the PO's role, but also with the architect's role. We think that only then, the RE field could provide value to agile large-scale distributed projects.

8. Conclusions

Admirable efforts have been done to investigate the challenges of agile RE [1,3–10,23,27,36,39,40,54]. However, previous publications have indicated the lack of empirical research in the challenging situations of QRs in particular in agile large-scale distributed context. Based on a multiple case study this paper contributes to the body of knowledge of agile RE by identifying 15 challenges agile practitioners cope with when dealing with QRs in agile large-scale distributed projects. Moreover, this study goes further by identifying 13 mechanisms behind those challenges and 9 practices agile teams currently use that could mitigate the impact of the challenges. Furthermore, we have mapped the identified mechanisms and practices to the reported challenges. Based on this mapping we have found that each mechanism can produce one or more challenges and each challenge can be the result of one or more mechanisms.

This study shows also that a particular practice could mitigate some challenges but also could introduce other challenges. Therefore we advise agile teams to carefully evaluate the used practices and get insight in the challenges that could be mitigated or introduced by the use of that particular practice. Only then agile teams could benefit optimally from the re-reported practices.

In this study we have further compared agile RE challenges which were previously identified (see Table 1) with the QRs challenges identified in this multiple case study. The paper shows that agile practitioners do not agree on what QRs mean and how to deal with them. Agile practitioners use the concept of user stories to express the customer desires which is – in the opinion of some of our participants, not equivalent to the concept of requirements. In addition, the paper indicates the lack of appropriate quality assurance practices to verify the right implementation of QRs.

Based on our findings, we want also to advise agile organizations to get insight in the maturity of their involved agile teams as early as possible in their projects. This insight will actually help an organization to define an appropriate communication model as described in [26]. Agile teams that lack maturity need a senior management role to coordinate the collaboration between the teams, while mature agile teams are self-organized teams, and would be less productive if a management role is involved to supervise their work.

This paper also shows that it is well possible that agile teams have the waterfall approach in their minds. Agile teams have to be careful when moving from waterfall to agile philosophy, since the wrong implementation of both will prevent the teams from gaining the benefits of either approach.

Supplementary material

Supplementary material associated with this article can be found, in the online version, at doi:10.1016/j.infsof.2019.01.009.

References

- [1] W. Alsaqaf, M. Daneva, R. Wieringa, Quality requirements in large-scale distributed agile projects - a systematic literature review, International Working Conference on Requirements Engineering: Foundation for Software Quality, 2017.
- [2] C. Larman, B. Vodde, Large-Scale Scrum More With Less, Pearson Education, 2016.
- [3] M. Käpyaho, M. Kauppinen, Agile requirements engineering with prototyping: a case study, in: RE2015, 2015, pp. 334–343.
- [4] K. Dikert, M. Paasivaara, C. Lassenius, Challenges and success factors for large-scale agile transformations: a systematic literature review, J. Syst. Softw. 119 (2016) 87–108.

- [5] R. Kasauli, G. Liebel, E. Knauss, S. Gopakumar, B. Kanagwa, Requirements engineering challenges in large-scale agile system development, in: 2017 IEEE 25th International Requirements Engineering Conference (RE), 2017, pp. 6–8.
- [6] K. Petersen, C. Wohlin, The effect of moving from a plan-driven to an incremental software development approach with agile practices: an industrial case study, *Empir. Softw. Eng.* 15 (2010) 654–693.
- [7] K.H. Rolland, “Desperately” seeking research on agile requirements in the context of large-scale agile projects, in: *Proc. XP2015*, 2015.
- [8] V. Sachdeva, L. Chung, Handling non-functional requirements for big data and IOT projects in Scrum, in: *Proc. 7th Int. Conf. Conflu. 2017 Cloud Comput. Data Sci. Eng.*, 2017, pp. 216–221.
- [9] B. Ramesh, L. Cao, R. Baskerville, Agile requirements engineering practices and challenges: an empirical study, *Inf. Syst. J.* 20 (2010) 449–480.
- [10] E. Bjarnason, K. Wnuk, B. Regnell, A case study on benefits and side-effects of agile practices in large-scale requirements engineering, in: *AREW*, 2011, pp. 1–5.
- [11] R.K. Yin, *Case Study Research Design and Methods*, Sage Publications Inc, 2013.
- [12] C. Boyce, P. Neale, Conducting in-depth interviews: a guide for designing and conducting in-depth interviews, *Evaluation* 2 (2006) 1–16.
- [13] D. Leffingwell, R. Knaster, *SAFe 4.0 Distilled: Applying the Scaled Agile Framework for Lean Software and Systems Engineering*, Pearson Education, 2017.
- [14] C. Larman, B. Vodde, *Practices For Scaling Lean & Agile Development*, Addison-Wesley Professional, 2010.
- [15] H. Kniberg, A. Ivarsson, *Scaling Agile @ Spotify - with Tribes, Squads, Chapters & Guilds*, 2012.
- [16] K. Charmaz, *Constructing Grounded Theory: A Practical Guide Through Qualitative Analysis*, 2006.
- [17] D. Hitchcock, The practice of argumentative discussion, *Argumentation* 16 (2002) 287–298.
- [18] J. Holvitie, S.A. Licorish, R.O. Spínola, S. Hyrynsalmi, V. Leppänen, Technical debt and agile software development practices and processes: an industry practitioner survey, *Inf. Softw. Technol.* 96 (2018) 141–160.
- [19] K. Schwaber, J. Sutherland, *The Scrum Guide*, 2016 <http://www.scrumguides.org/index.html>.
- [20] P. Mohagheghi, M.E. Aparicio, An industry experience report on managing product quality requirements in a large organization, *Inf. Softw. Technol.* 88 (2017) 96–109.
- [21] W. Alsaqaf, M. Daneva, R. Wieringa, Understanding challenging situations in agile quality requirements engineering and their solution strategies: insights from a case study, *RE2018*, 2018.
- [22] C. Mario, *Executable Specifications with Scrum*, Pearson Education, 2013.
- [23] E. Schön, D. Winter, M.J. Escalona, Key challenges in agile requirements engineering, in: *International Conference on Agile Software Development*, 283, 2017, pp. 37–51.
- [24] P.N. Adler, *Membership Roles in Field Research*, SAGE Publications Inc, 1987.
- [25] N. King, C. Horrocks, *Interviews in Qualitative Research*, SAGE Publications Ltd, 2010.
- [26] S. Lauesen, *Software Requirements: Style and Techniques*, Pearson Education, 2002.
- [27] T. Dingsøyr, N.B. Moe, T.E. Fægri, E.A. Seim, Exploring software development at the very large-scale: a revelatory case study and research agenda for agile method adaptation, *Empir. Softw. Eng.* 23 (1) (2017).
- [28] K.H. Rolland, B. Fitzgerald, T. Dingsøyr, K.-J. Stol, Problematising Agile in the Large: Alternative Assumptions For Large-Scale Agile Development, *ICIS*, 2016.
- [29] D. Domah, F.J. Mitropoulos, The NERV methodology: a lightweight process for addressing non-functional requirements in agile software development, *Southeastcon*, 2015.
- [30] Jørgensen, M., Do agile methods work for large software projects? *XP 2018*: 179–190
- [31] I. Bensabat, D.K. Goldstein, M. Mead, The case research strategy in studies of information systems, *MISQ* 11 (3) (1987) 369–386.
- [32] S.B. Merriam, *Qualitative Research and Case Study Applications in Education*, Jossey-Bass Publishers, 1998.
- [33] M. Daneva, A. Herrmann, L. Buglione, Coping with quality requirements in large, contract-based projects, *IEEE Softw.* 32 (6) (2015) 84–91.
- [34] Daneva, M., Marczak, S., Herrmann, A., Engineering of quality requirements as perceived by near-shore development centers’ architects in eastern Europe: the hole in the whole, *ESEM 2014*: 19:1–19:10
- [35] Y.I. Alzoubi, A.Q. Gill, A. Al-Ani, Empirical studies of geographically distributed agile development communication challenges: a systematic review, *Inf. Manag.* 53 (1) (2016) 22–37.
- [36] I. Inayat, S. Salwah Salim, S. Marczak, M. Daneva, S. Shamshirband, A systematic literature review on agile requirements engineering practices and challenges, *Comput. Hum. Behav.* 51 (2015) 915–929.
- [37] S.W. Ambler, M. Lines, *Disciplined Agile Delivery: A Practitioner’s Guide to Agile Software Delivery in the Enterprise*, IBM Press, 2012.
- [38] W. Alsaqaf, M. Daneva, R. Wieringa, Quality requirements challenges in the context of large-scale distributed agile: an empirical study, in: *REFSQ2018*, 2018, pp. 1–16.
- [39] W. Behutiye, P. Karhapää, D. Costal, M. Oivo, X. Franch, Non-functional requirements documentation in agile software development: challenges and solution proposal, in: *International Conference on Product-Focused Software Process Improvement*, in: *Lect. Notes Comput. Sci. (including Subser. Lect. Notes Artif. Intell. Lect. Notes Bioinformatics)*, vol. 10611, LNCS, 2017, pp. 515–522.
- [40] F.G. De Oliveira Neto, J. Horkoff, E. Knauss, R. Kasauli, G. Liebel, Challenges of aligning requirements engineering and system testing in large-scale agile: a multiple case study, in: *Proc. - 2017 IEEE 25th Int. Requir. Eng. Conf. Work. REW 2017*, 2017, pp. 315–322.
- [41] M. Kassab, O. Ormandjieva, M. Daneva, An ontology based approach to non-functional requirements conceptualization, in: *ICSEA 2009*, 2009, pp. 299–308.
- [42] T.E. Fægri, N.B. Moe, Re-conceptualizing requirements engineering: findings from a large-scale, agile project, in: *XP2015 - XP ’15 Work.*, 2015, pp. 1–5.
- [43] C.M. Robert, M. Micah, *Agile Principles, Patterns and Practices in C#*, Prentice Hall, 2006.
- [44] Agile Alliance, *Manifesto for Agile Software Development*, 2001.
- [45] J. Apperlo, *Management 3.0 Leading Agile Developers, Developing agile Leaders*, Pearson Education, 2011.
- [46] R. Davies, Non-Functional Requirements: Do User Stories Really Help?, 2009 *DevOpsDays[Online]*. Available: <http://www.methodsandtools.com/archive/archive.php?id=113>.
- [47] J.D.R.V. Medeiros, D.C.P. Alves, A. Vasconcelos, C. Silva, E. Wanderley, Requirements engineering in agile projects: a systematic mapping based in evidences of industry, in: *CibSE*, 2015, pp. 460–473.
- [48] J. Conklin, Dialog mapping: reflections on an industrial strength case study, *Vis. Argum.* (2003) 1–15.
- [49] M.A. Babar, P. Kruchten, P. Abrahamsson, Agility and architecture: can the coexist? *IEEE Softw.* 27 (2) (2010) 16–22.
- [50] D. Leffingwell, *Agile Software Requirements: Lean Requirements Practices for Teams, Programs, and the Enterprise*, First ed., Pearson Education, 2011.
- [51] M. Daneva, A. Herrmann, L. Buglione, Coping with quality requirements in large, contract-based projects, *IEEE Softw.* (6) (2015) 84–91.
- [52] M. Daneva, et al., Agile requirements prioritization in large-scale outsourced system projects: an empirical study, *J. Syst. Softw.* 86 (5) (2013) 1333–1353.
- [53] J. Nawrocki, M. Ochodek, J. Jurkiewicz, S. Kopczyńska, B. Alchimowicz, Agile requirements engineering: a research perspective, in: *SOFSEM*, 8327, 2014, pp. 40–51.
- [54] Uludag, O., Kleeaus, M., Caprano, C., Matthes, F., Identifying and structuring challenges in large-scale agile development based on a structured literature review, *EDOC’18*, pp. 191–197.